CAM2003C - Data Structures and Algorithms with C and C++

Lab Exercise -8: Tree and Heaps

Practical Questions on Tree and Heaps

Objective

To implement and understand the operations of Tree Data Structures (Binary Tree, Threaded Binary Tree, Binary Search Tree, AVL Tree and Heaps) and their applications.

Perform the following in C/ C++.

Binary Tree

1. Write a program to implement a **Binary Tree** using an **Array**. Include the following operations:
    - Insert elements into the binary tree.
    - Display the binary tree as an array.
    - Perform **In-order**, **Pre-order**, and **Post-order Traversals** using the array.

    - **Instructions:** Use a 1D array to store the binary tree where the left child of a node at index i is stored at 2*i + 1 and the right child at 2*i + 2. Test with different inputs to insert elements and perform traversals.
2. Implement a binary tree from scratch in C or C++. Include functions to:
    - Insert a node into the tree.
    - Display the tree using **Pre-order**, **In-order**, and **Post-order** traversals.
    - Find the **height** of the tree.
    - Count the **total number of nodes**.
    - Count the **number of internal nodes**.
    - Count the **number of leaf nodes (External nodes)**.

    - **Instructions:** Write separate functions for each of the above tasks. Test your implementation with a sample tree.
3. Implement a function to compute the **Mirror Image** of a **Binary Tree**

4. Implement a program to perform a **Level-order Traversal** of a binary tree that is stored in an **Array**.

- **Instructions:** Given an array representing a binary tree, print the elements level by level. Use the array index to determine the children of each node as described in the previous exercise.

5. Write a program to check if a given **Binary Tree** stored in an **Array** is a **Complete Binary Tree**.
   - **Instructions:** Use the properties of a complete binary tree to verify if the array representation satisfies the conditions. Test the function with various array inputs.

6. Write a program to implement a **Binary Tree** using **Linked Lists**. The tree should support the following operations:
      - Insert a node into the binary tree.
      - Delete a node.
      - Perform **In-order**, **Pre-order**, and **Post-order Traversals**.
   - **Instructions:** Create a binary tree where each node has left and right pointers. Write functions to insert nodes and perform different types of tree traversals.

Expression Tree

1. Write a program to construct an **Expression Tree** for a given postfix expression. Implement the following:
      - Construct the expression tree.
      - Perform **In-order**, **Pre-order**, and **Post-order** traversals of the tree to display the expression in infix, prefix, and postfix notations.
   - **Instructions:** Test the program using a sample postfix expression such as ab+cd-*.

Threaded Binary Tree

1. Write a program to Implement a Threaded Binary Tree (One Way Threading)
   - Implement a left-threaded **Binary Tree** in C or C++.
   - Implement a right-threaded **Binary Tree** in C or C++.

2. Implement a **Doubly Threaded Binary Tree** where each node has two threads—one for the in-order predecessor and one for the in-order successor. Include the following operations:
      - Insert a node while maintaining both threading links.
      - In-order traversal using the threading.
   - **Instructions:** Test your implementation with sample data and ensure that both forward and reverse traversals are correct.

3. Write a program to convert a given **Binary Tree** into a **Threaded Binary Tree**. Include the following:
    - Convert the binary tree into a left-threaded or right-threaded binary tree.
    - Perform In-order traversal using the threaded links.
    - **Instructions:** Use recursive or iterative methods to convert the tree. After conversion, test the threaded tree by performing the traversal without recursion.

Binary Search Tree

1. Implement a **Binary Search Tree (BST)** with the following operations:
    - Insert a node.
    - Delete a node.
    - Search for a node.
    - Find the minimum and maximum elements.
    - Perform In-order, Pre-order, and Post-order traversals.
    - **Instructions:** Create a menu-driven program where users can choose which operation to perform.

AVL Tree

1. Implement an **AVL Tree** and perform the following operations:
    - Insert a node with auto-balancing.
    - Perform left, right, left-right, and right-left rotations to maintain balance.
    - Display the tree's In-order, Pre-order, and Post-order traversals.

Heaps

1. Implement a **Max Heap** or **Min Heap**. Include the following operations:
    - Insert a node.2
    - Delete the root node.
    - Display the heap as a binary tree.
    - Perform **Heap Sort** using the heap structure.
    - **Instructions:** Demonstrate the working of the heap with sample data and show the sorted array using heap sort.
2. Write a function to **check if a given array** represents a valid **Max Heap**.

- **Instructions:** The function should traverse the array and verify if the heap property (i.e., parent node is greater than its children) is satisfied at every node. Return true if it is a Max Heap; otherwise, return false.

3. Write a program to **delete an element** from a Max Heap. After deletion, re-heapify the heap to maintain the heap property.
   - **Instructions:** Provide an interface to allow the user to specify which element to delete, perform the deletion, and display the updated heap.
4. Implement a function to **build a Max Heap** from an unsorted array **Heapify Operation)**. Perform the following operations:
   - Insert elements into the heap.
   - Display the array as a Max Heap.
   - **Instructions:** Use the **bottom-up** approach to build the Max Heap, where heapify is applied starting from the non-leaf nodes. Test with an unsorted array.
5. Implement **Heap Sort** using a **Max Heap**. Include the following steps:
   - Build a Max Heap from the given array.
   - Perform Heap Sort to sort the array in ascending order.
   - **Instructions:** The program should first construct a Max Heap, then repeatedly remove the largest element (the root) and re-heapify the remaining elements. Display the sorted array.
6. Implement **Heap Sort** using a **Min Heap**. Include the following steps:
   - Build a Min Heap from the given array.
   - Perform Heap Sort to sort the array in **descending order**.
   - **Instructions:** The program should first construct a Min Heap, then repeatedly remove the smallest element and re-heapify. Display the sorted array in descending order.
7. Implement a **Priority Queue** using a **Max Heap**. The following operations should be supported:
   - **Insert** an element into the queue.
   - **Extract the maximum** element (highest priority).
   - **Peek** at the maximum element without extracting it.
   - **Increase key** of a given element.
   - **Instructions:** Implement the priority queue as a Max Heap, where the element with the highest priority is always at the root. Provide a menu-driven interface for performing the above operations.
8. Implement a **Priority Queue** using a **Min Heap**. The following operations should be supported:
   - **Insert** an element into the queue.

    - **Extract the minimum** element (highest priority in Min Heap).
    - **Peek** at the minimum element without extracting it.
    - **Decrease key** of a given element.
- **Instructions:** Implement the priority queue as a Min Heap, where the element with the lowest priority (highest priority in Min Heap) is at the root. Test with a sample set of priorities.